Single-stream Policy Optimization

- Purpose of this new framework; why is this framework better than GRPO?
  - Group-Free for Scalable Infrastructure.
    - The traditional GRPO needs to sample several groups with Gsamples and calculate the advantage within each group.
    - In distributed GRPO training, the efficiency of each group is limited by the slowest sample in that group, which is a bottleneck.
    - A group-independent framework is needed.
  - Eliminates Degenerate Groups
    - In GRPO, some input prompt $x$will cause the answers in a group to be very similar (especially in binary problems: all correct or all wrong), which will cause the advantage in that group ~0: **the model can literally learn nothing.**
    - A framework to avoid this problem is needed, in which the signal is not group-based or temporary and can always produce a useable gradient.
  - More Stable and Low-Variance Signal: GRPO is group-based, so the variance between each group can be very large, producing noises and becoming less stable.
- Methodologies *(for binary tasks, right-or-wrong)*
  - PPO is a good start to single-stream policy optimization; however, a light-weight value estimator is needed.
  - In this case, we can model the success probability $\hat{v}(x)$using a Beta distribution:

$\hat{v}(x) \sim \mathrm{Beta}(\alpha(x), \beta(x))$,

And the value will be estimated by:

$\hat{v}(x) = \frac{\alpha(x)}{\alpha(x)+\beta(x)}$,

where α is the discounted effective count of successes (for x) and β the corresponding failures.

  - Initialization
    1. Using a sample with size $n_0$to estimate an initial $\hat{v}_0$.
    2. Determine an efficient initial sample size $N_0$by $N_0 = \frac{1}{1-\rho_{min}}$, where $\rho_{min}$is the minimum allowed forgetting factor.
    3. Then init $\alpha_0$and $\beta_0$by:

$\alpha_0 = N_0 \cdot \hat{v}_0(x)$and $\beta_0 = N_0 \cdot (1 - \hat{v}_0(x))$

    - The algorithm is like:

Algorithm 2 SPO Initialization

```
Set initial effective sample size N_0 = 1/(1 − ρ_min).
for each prompt x ∈ 𝒳 do
    Collect n_0 outcomes {r^(k)}_{k=1}^{n_0} with an initial policy π_0.
    Compute initial value estimate v̂_0(x) = (1/n_0) Σ_{k=1}^{n_0} r^(k).
    Set α_0(x) = N_0 · v̂_0(x) and β_0(x) = N_0 · (1 − v̂_0(x)).
```

  - Updating:
    - Update of $\alpha$and β

$\alpha(x) = \rho(x)\alpha_{-1}(x) + r(x, y)$and $\beta(x) = \rho(x)\beta_{-1}(x) + (1 - r(x. y))$, where

- $\cdot_{-1}$is the previous value
- $r(x, y)$, the reward, $\in [0, 1]$for binary tasks
- $\rho(x) = 2^{-\frac{D(x)}{D_{half}}}$, is the discount factor. $D(x)$is the KL divergence, and $D_{half}$is a hyperparameter to control the forgotten rate $\rho \in [\rho_{min}, \rho_{max}]$. When the new and old policy models differ more, meaning a larger KL divergence, ρwill become smaller, so $\alpha_{-1}$and β_{-1}will have a smaller weight, resulting in more radical updating of the value estimator.

- Equivalently, this can be represented in *Exponential Moving Average* form:

  $\hat{v}(x) = \hat{v}_{-1}(x) + \eta(x)(r(x, y) - \hat{v}_{-1}(x))$,

  where $\eta(x) = (\rho(x)N_{eff,-1}(x) + 1)^{-1}$ and $N_{eff} = \alpha(x) + \beta(x) + 1$

- Advantage
  - Similar to that in PPO, here:

    $A(x, y) = r(x, y) - \hat{v}_{-1}(x)$

  - Normalization: we calculate the mean $\mu_{\mathrm{B}}$and stdv $\sigma_{\mathrm{B}}$of the advantage across *all prompts* in that batch, and:

    $\widetilde{A}(x, y) = \frac{A(x,y) - \mu_{\mathrm{B}}}{\sigma_{\mathrm{B}}}$

  - Substitude the $\widetilde{A}$to the PPO's objective:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_{s,t}\left[\min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}\tilde{A}_t, \mathrm{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}, 1-\varepsilon, 1+\varepsilon\right)\tilde{A}_t\right)\right]$$

- Combine these together, the algorithm will be:

Algorithm 1 Single-stream Policy Optimization

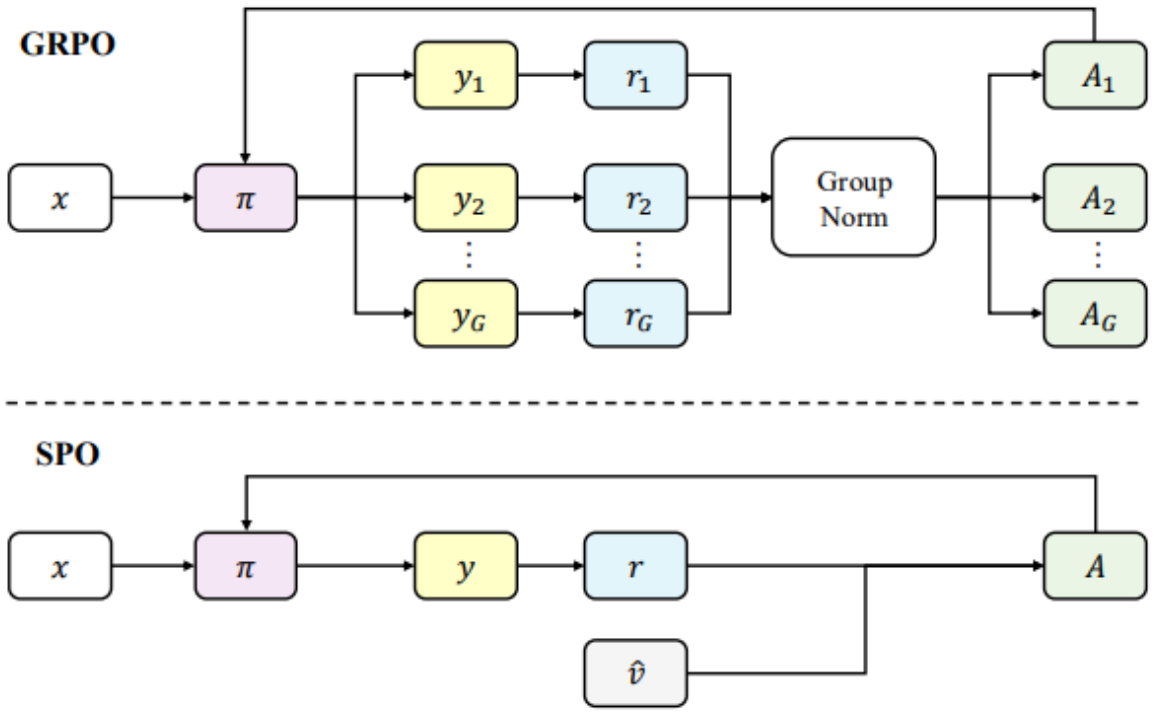
```
for iteration i = 1, 2, ..., T do
    For each x ∈ 𝒳, compute sampling weight w_i(x) according to Eqn. (11).
    Sample a batch of B prompts B_i ⊂ 𝒳 according to weights {w_i(x)}.
    𝒟 ← ∅
    for each prompt x ∈ B_i do
        Sample action y ~ π_{θ_{i-1}}(· | x) and observe reward r(x, y) ∈ {0, 1}.
        Compute raw advantage A(x, y) ← r(x, y) − v̂_{-1}(x).
        Store (x, y, A(x, y)) in 𝒟.
        Update tracker v̂(x).
    Normalize advantages: Ã(x, y) ← (A(x, y) − μ_{B_i})/σ_{B_i}.
    Update θ_{i-1} to θ_i using mini-batches with a policy gradient algorithm (e.g., PPO-Clip).
```

- Data Sampling: we give each prompt a weight by

$$w_i(x) \propto \sqrt{\hat{v}_{-1}(x)(1 - \hat{v}_{-1}(x))} + \epsilon$$

In this case, the problems with neither "easy peasy" nor "very tough" will have a larger weight (50%-50% will achieve the maximum, most unsure).

- SPO vs. GRPO:



- Extend to non-binary tasks: ignore all $\alpha(x)$and $\beta(x)$terms.
  - During initialization, ignore line 5 of *algorithm 2*, directly use $\hat{v}_0(x)$.
  - During updating, use *EMA form*.
  - *(The non-binary task related experiments were never conducted in the original paper. Thus, the actual performance of the framework in non-binary tasks is still unknown.)*
- Work Cited and Chat History
  - Paper
  - ChatGPT
  - Gemini